

TDVI: Inteligencia Artificial

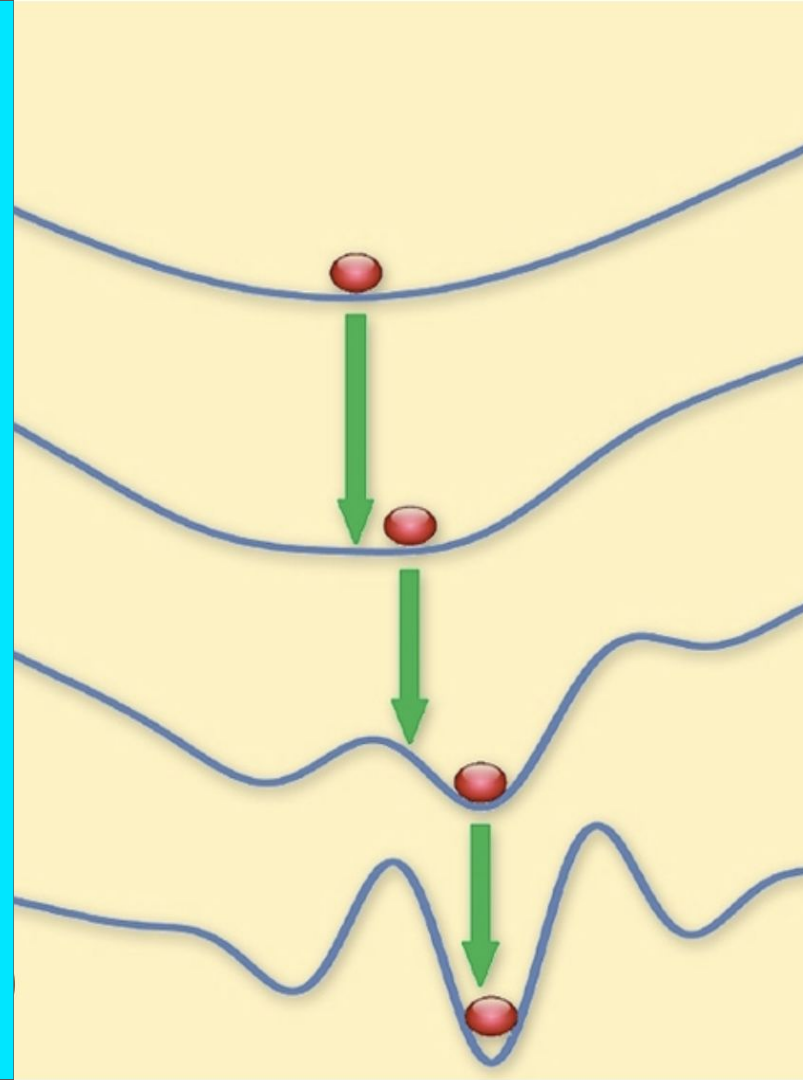
Optimizadores

Licenciatura en Tecnología Digital



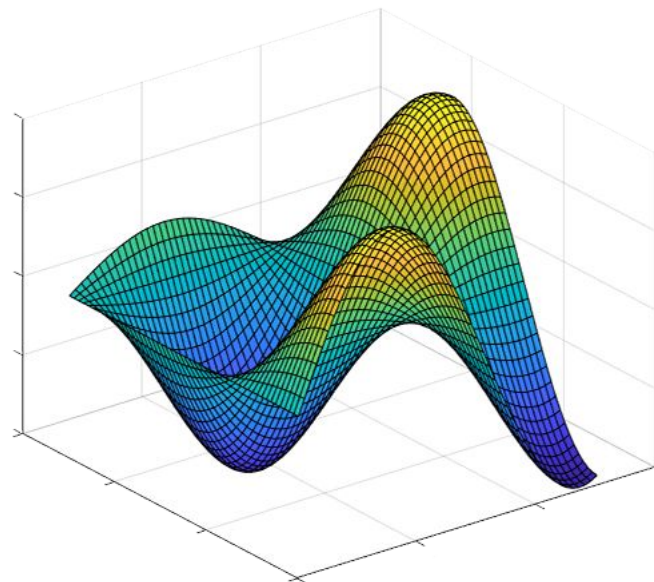
Agenda

1. **Optimizadores basados en cálculos de gradiente**
2. Error Landscape
3. Learning Rate
4. Schedules
5. Estrategias de actualización de pesos
6. Acelerando convergencias
7. Learning rate adaptativo
8. ¿Cómo elegir?
9. Bibliografía



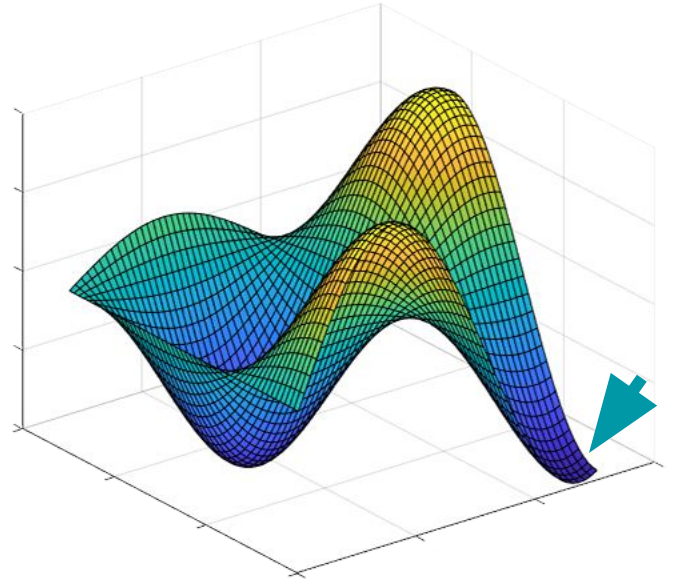
La geometría del error

- Los algoritmos de deep learning requieren optimizaciones.
 - Es decir, minimizar una función de error.
 - Podemos pensar los errores de nuestro modelo en términos geométricos.



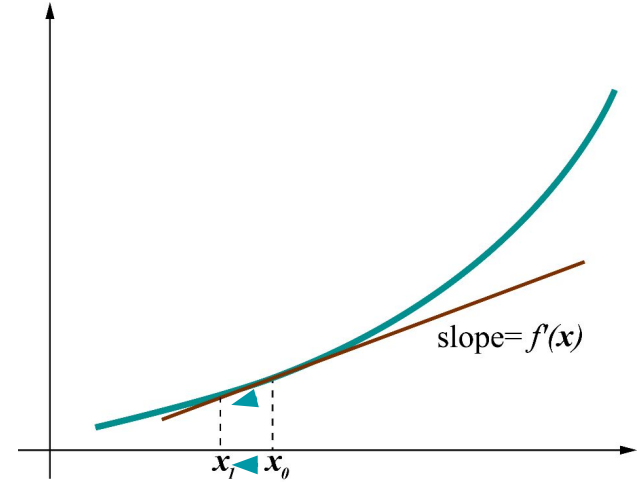
Función Objetivo

- La función que minimizamos es la función objetivo / función de pérdida o función de costo.
 - $\mathbf{x}^* = \arg \min \mathbf{f}(\mathbf{x})$
- $f'(\mathbf{x})$ nos da la pendiente en el punto $\mathbf{x}$
 - la dirección de cambio que necesitamos para mejorar nuestro output
 - $\mathbf{f}(\mathbf{x} + \varepsilon) \approx \mathbf{f}(\mathbf{x}) + \varepsilon \mathbf{f}'(\mathbf{x})$



Estrategia de minimización

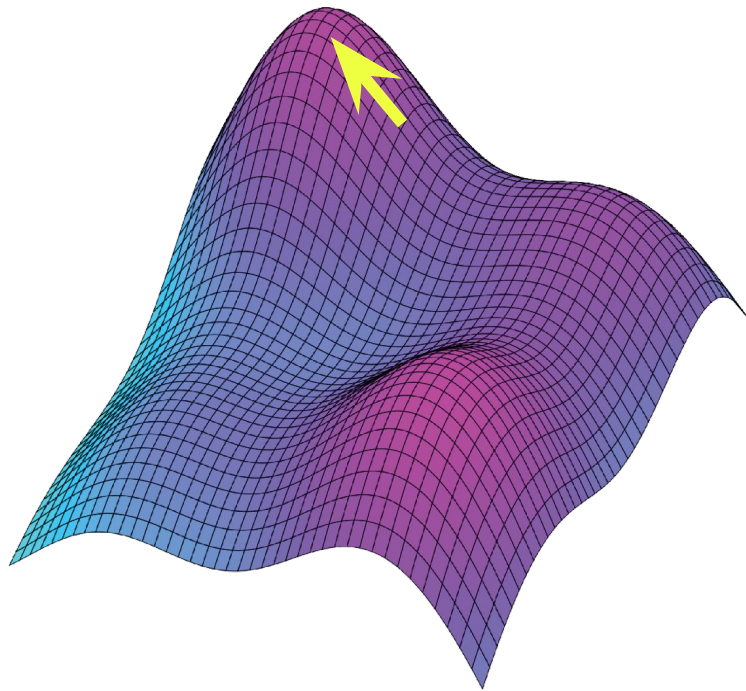
- Utilizamos una estrategia para minimizar la función objetivo
- Podemos encontrar el x que minimiza $f(x)$ actualizando x de a pequeños pasos **en dirección opuesta a la derivada**.
- Utilizamos el término **gradiente** para funciones con múltiples inputs



Vector Gradiente

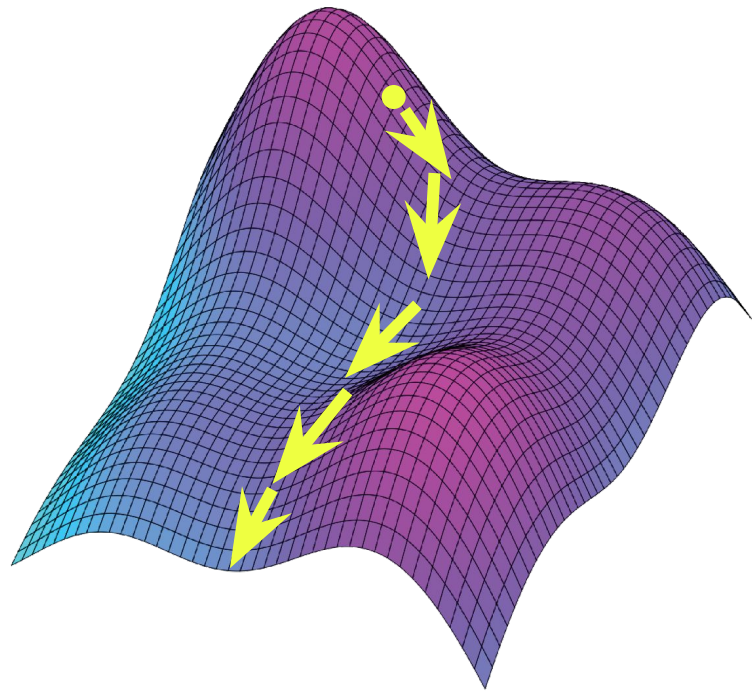
- Cuando hay varios inputs, generalizamos la derivada.
- Es un vector con derivadas parciales

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \cdot \\ \cdot \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{bmatrix}$$



Descente del gradiente

- Gradient descent (Cauchy, 1847)
- Actualizamos el punto x utilizando la dirección opuesta al gradiente (downhill)
 - $x_{t+1} = x_t - \varepsilon \nabla f(x_t)$
- Secuencia monótona con ε chiquito
 - $f(x_t) \geq f(x_{t+1}) \geq f(x_{t+2}) \geq \dots$
- Converge cuando el gradiente se aproxima a cero (≈ 0)

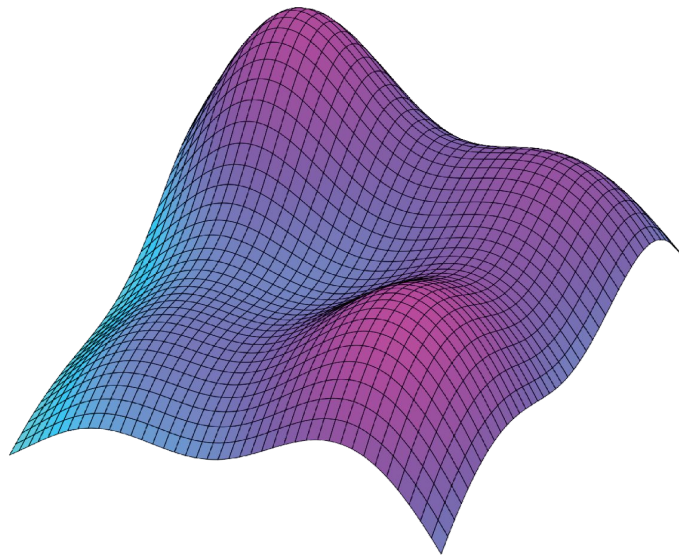


Agenda

1. Optimizadores basados en cálculos de gradiente
2. **Error Landscape**
3. Learning Rate
4. Schedules
5. Estrategias de actualización de pesos
6. Acelerando convergencias
7. Learning rate adaptativo
8. ¿Cómo elegir?
9. Bibliografía

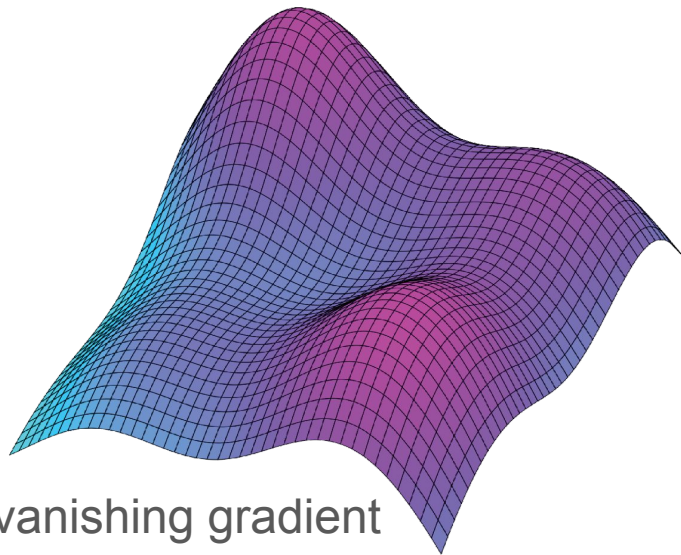
Formas características

- Mínimo
- Máximo
- Llanura (Plateau)
- Silla de montar (saddle)



Formas características

- Mínimo
- Máximo
- Llanura (Plateau)
- Silla de montar (saddle)

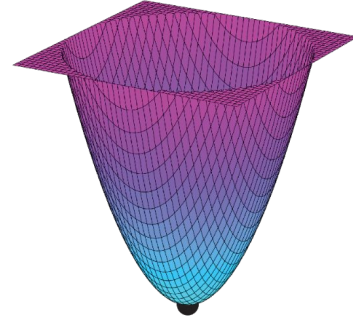


Todos estos casos generan el problema del vanishing gradient

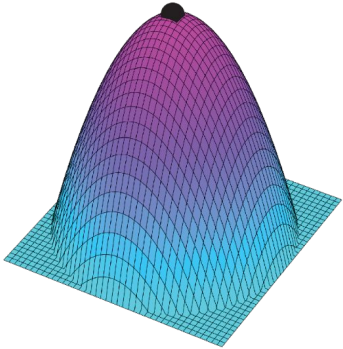
- gradiente es cero
- los pesos son cero
- no mejoramos la performance del modelo

Mínimo y Máximo

- El **mínimo** es un punto de la superficie donde el **gradiente es cero** y movimientos en cualquier dirección nos mueven para **arriba**.

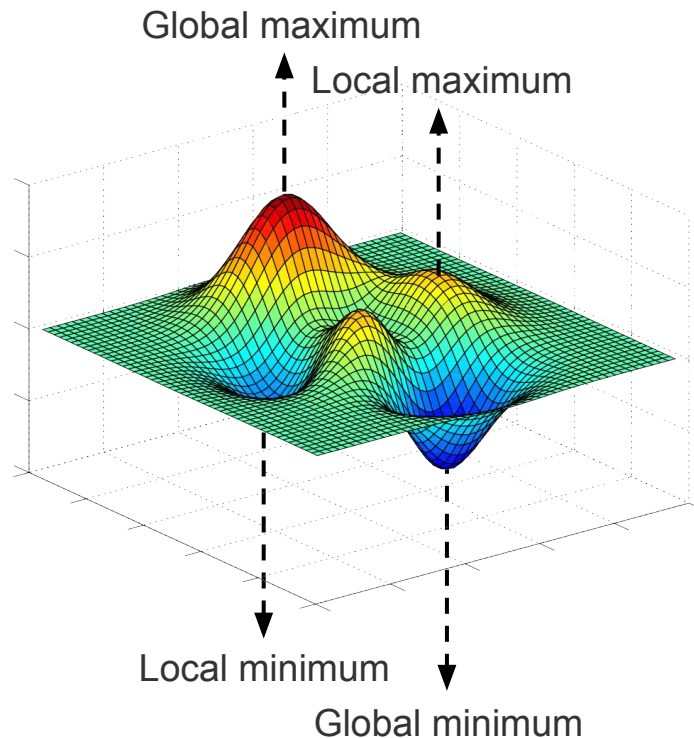


- El **máximo** es un punto de la superficie donde el **gradiente es cero** y movimientos en cualquier dirección nos mueven para **abajo**.



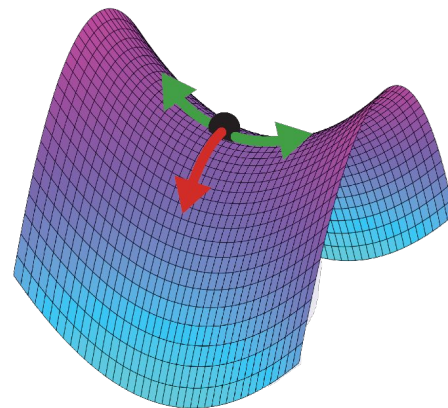
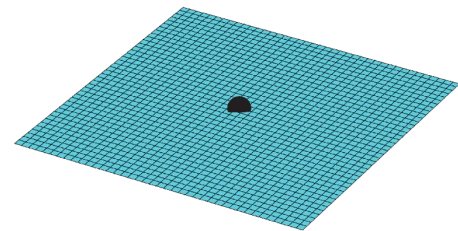
Global vs. Local

- Las funciones de error pueden tener múltiples mínimos y máximos locales
- El punto con el valor mínimo/máximo **absoluto** de $f(x)$ es el mínimo/máximo global



Llanura y silla de montar

- La llanura (**plateau**) es donde la superficie es plana, llana, y no hay cambios en altura hacia ninguna dirección.
 - El gradiente es cero, es decir, no hay dirección de cambio.
- En un **saddle (silla de montar)**, los desplazamientos en uno o más ejes resulta en un aumento del error, **y a su vez**, desplazamientos en uno o más ejes resultan en decremento del error.



Loss Landscape

Visualizing the Loss Landscape of Neural Nets

Hao Li¹, Zheng Xu¹, Gavin Taylor², Christoph Studer³, Tom Goldstein¹

¹University of Maryland, College Park ²United States Naval Academy ³Cornell University
{hao1i,xuzh,tong}@cs.umd.edu, taylor@usna.edu, studer@cornell.edu

Abstract

Neural network training relies on our ability to find “good” minimizers of highly non-convex loss functions. It is well-known that certain network architecture designs (e.g., skip connections) produce loss functions that train easier, and well-chosen training parameters (batch size, learning rate, optimizer) produce minimizers that generalize better. However, the reasons for these differences, and their effects on the underlying loss landscape, are not well understood. In this paper, we explore the structure of neural loss functions, and the effect of loss landscapes on generalization, using a range of visualization methods. First, we introduce a simple “filter normalization” method that helps us visualize loss function curvature and make meaningful side-by-side comparisons between loss functions. Then, using a variety of visualizations, we explore how network architecture affects the loss landscape, and how training parameters affect the shape of minimizers.

1 Introduction

Training neural networks requires minimizing a high-dimensional non-convex loss function – a task that is hard in theory, but sometimes easy in practice. Despite the NP-hardness of training general neural loss functions [2], simple gradient methods often find global minimizers (parameter configurations with zero or near-zero training loss), even when data and labels are randomized before training [42]. However, this good behavior is not universal; the trainability of neural nets is highly dependent on network architecture design choices, the choice of optimizer, variable initialization, and a variety of other considerations. Unfortunately, the effect of each of these choices on the structure of the underlying loss surface is unclear. Because of the prohibitive cost of loss function evaluations (which requires looping over all the data points in the training set), studies in this field have remained predominantly theoretical.

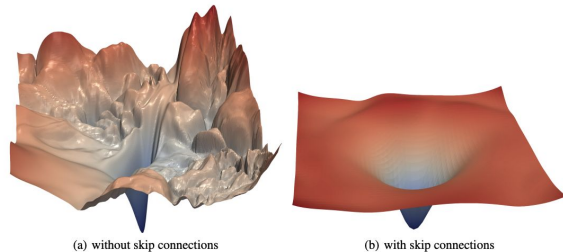
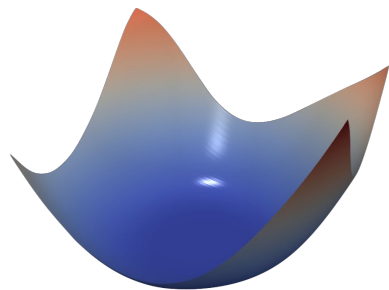
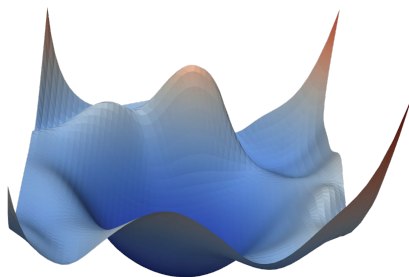


Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures. 32nd Conference on Neural Information Processing Systems (NIPS 2018), Montréal, Canada.



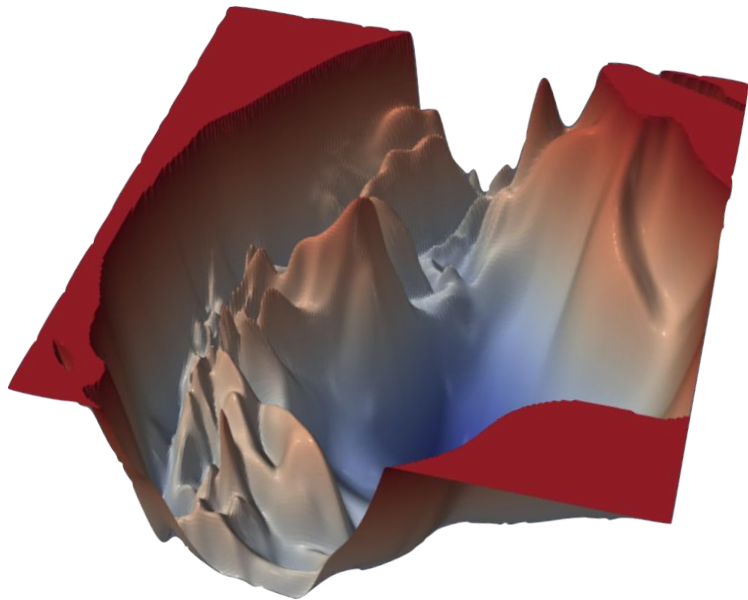
Densenet-121



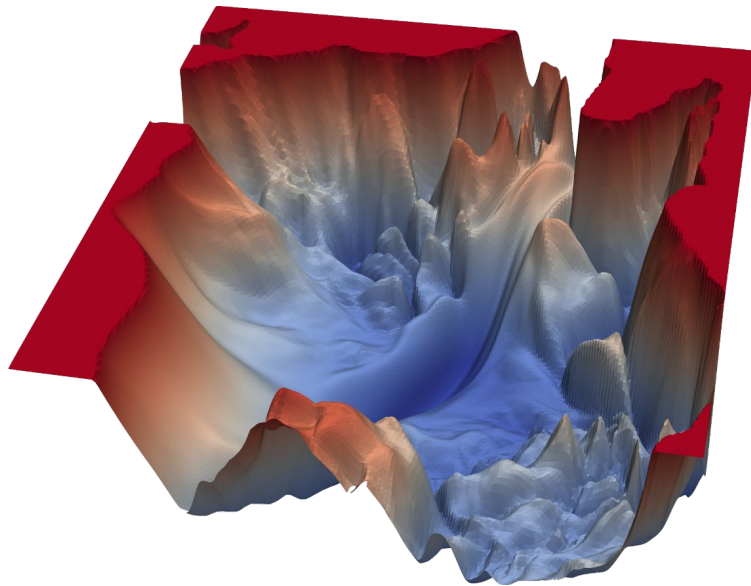
Resnet-56

arXiv:1712.09913v3 [cs.LG] 7 Nov 2018

Loss Landscape (cont.)



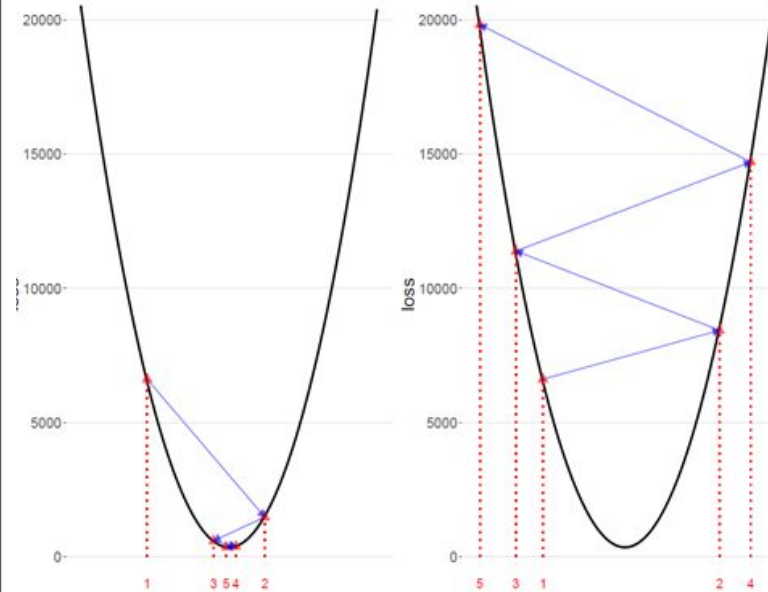
VGG-110



VGG-56

Agenda

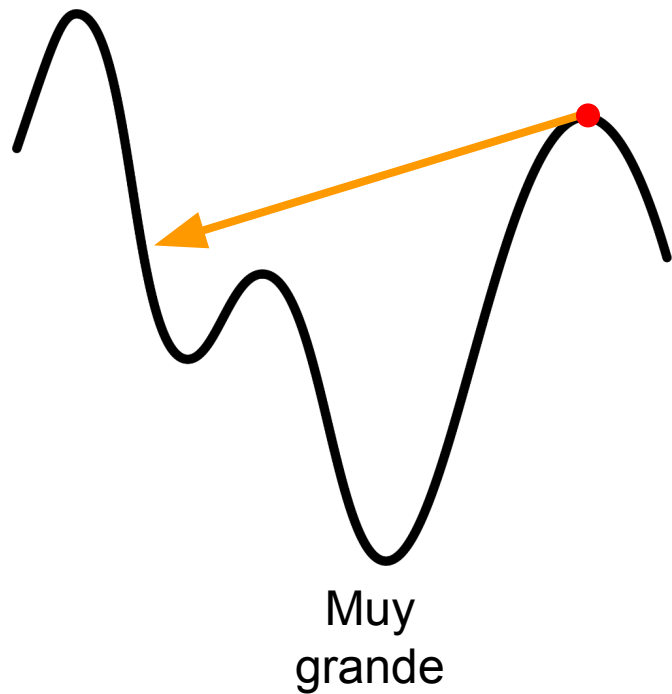
1. Optimizadores basados en cálculos de gradiente
2. Error Landscape
3. **Learning Rate**
4. Schedules
5. Estrategias de actualización de pesos
6. Acelerando convergencias
7. Learning rate adaptativo
8. ¿Cómo elegir?
9. Bibliografía



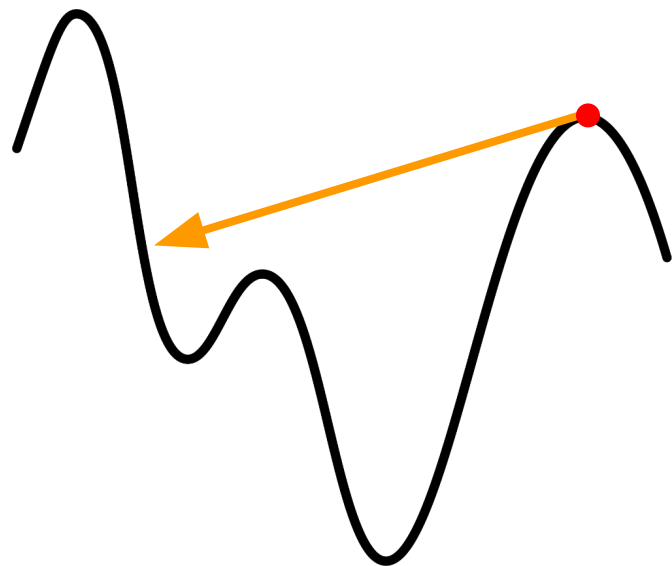
Learning Rate

- Regla de actualización del descenso del gradiente
 - $x_{t+1} = x_t - \varepsilon \nabla f(x_t)$
- **Learning rate** (ε) es un hyper parámetro crítico
 - Determina la proporción del gradiente para actualizar los pesos

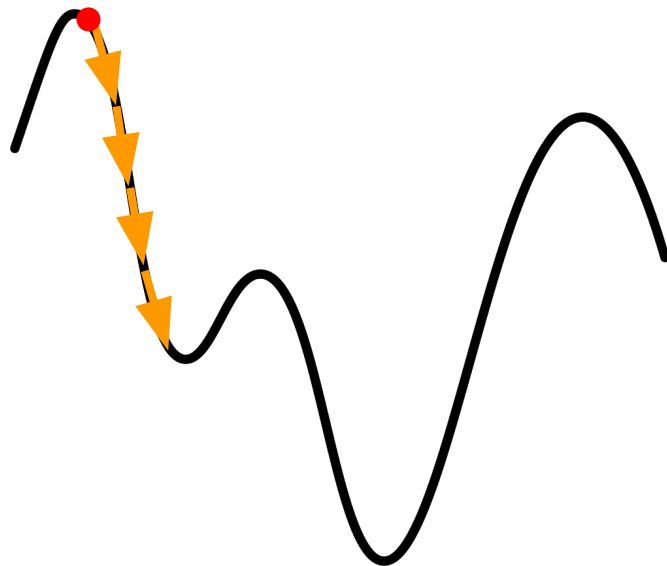
Learning Rate (cont.)



Learning Rate



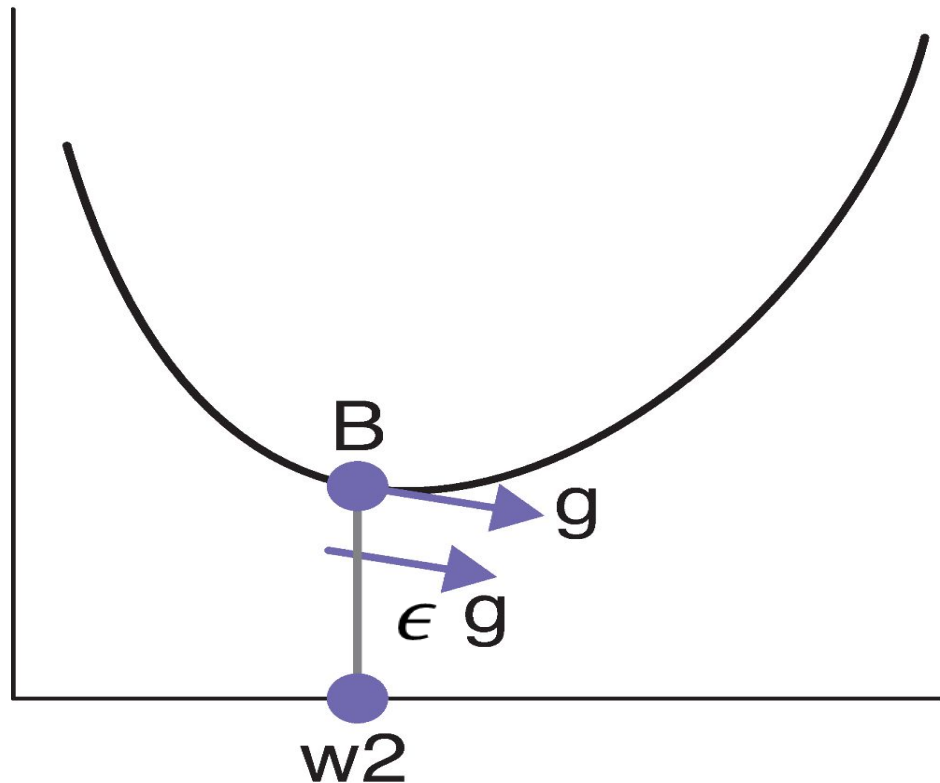
Muy
grande



Muy
chico

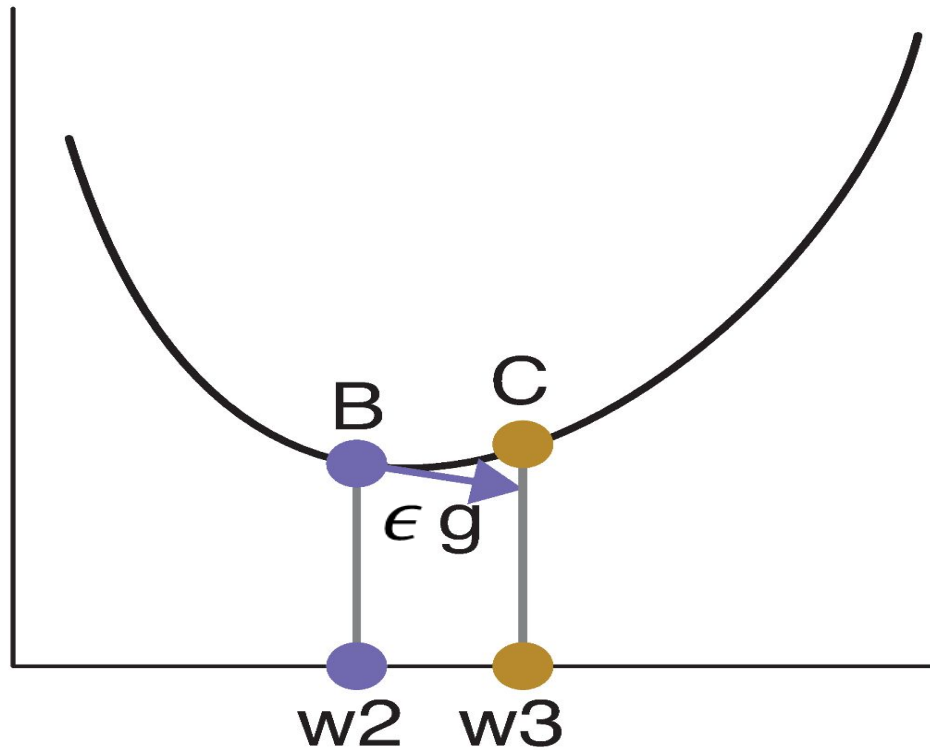
Trayectoria en el espacio de parámetros

- La trayectoria que tomamos está definida por el espacio de los parámetros de la red (pesos)

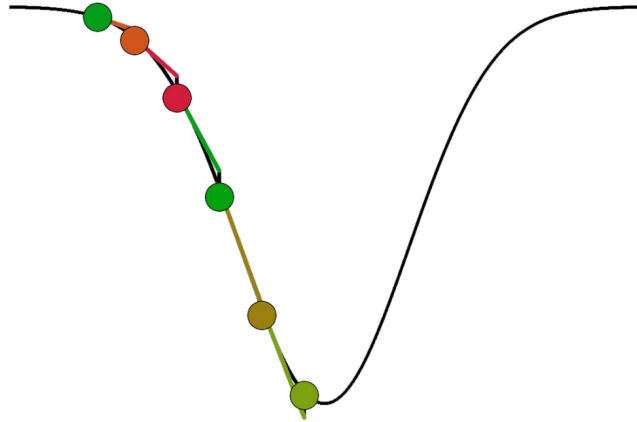


Trayectoria en el espacio de parámetros

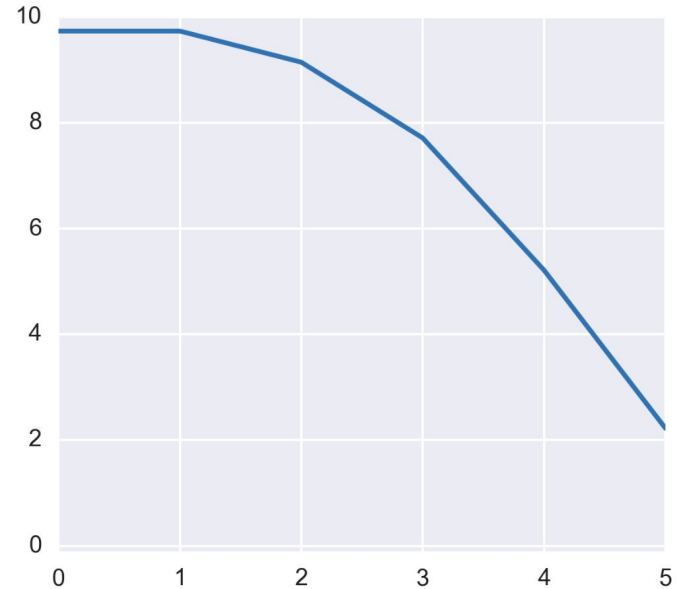
- La trayectoria que tomamos está definida por el espacio de los parámetros de la red (pesos)
- Para encontrar w_3 (el nuevo valor del peso) sumamos el gradiente escalado (multiplicado por ϵ) a w_2



Learning Rate [LR] (tasa de aprendizaje) Constante

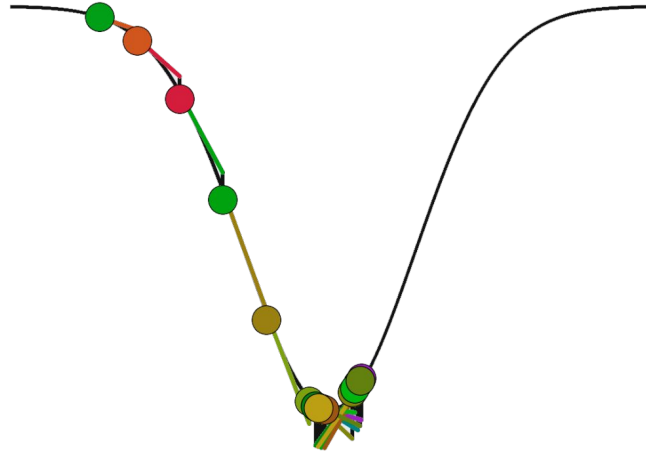


LR $\epsilon = 1/8$, 6 pasos

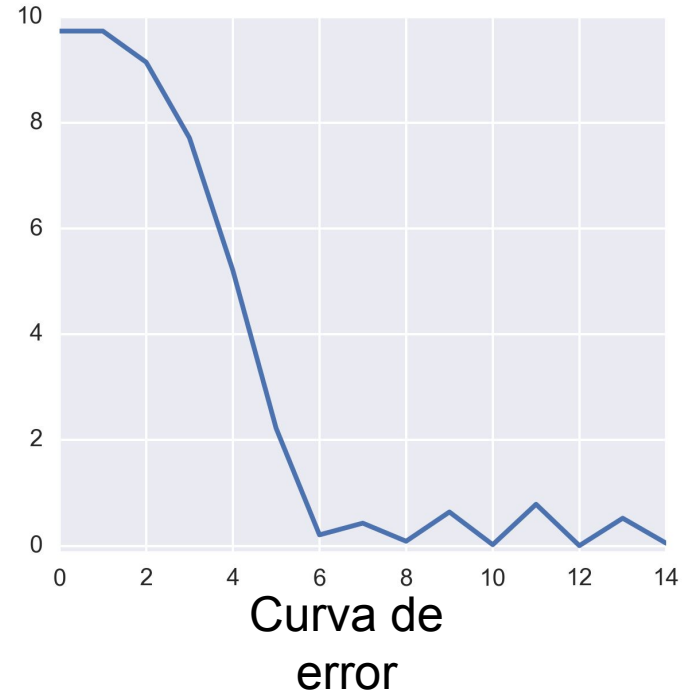


Curva de
error

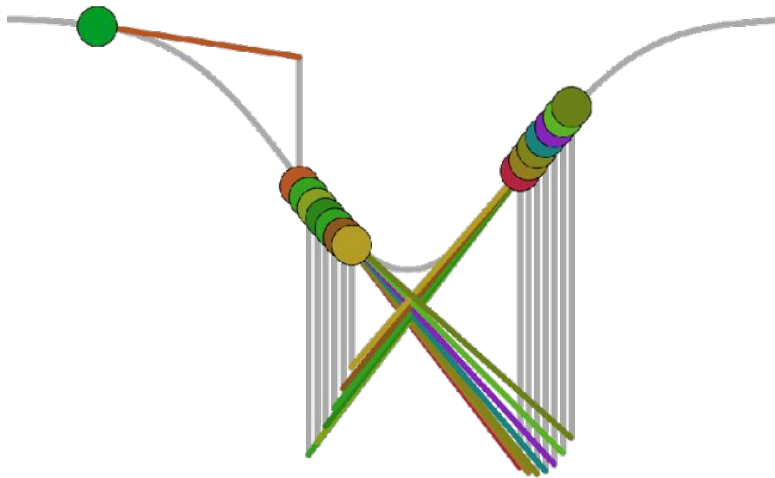
Learning Rate [LR] (tasa de aprendizaje) Constante



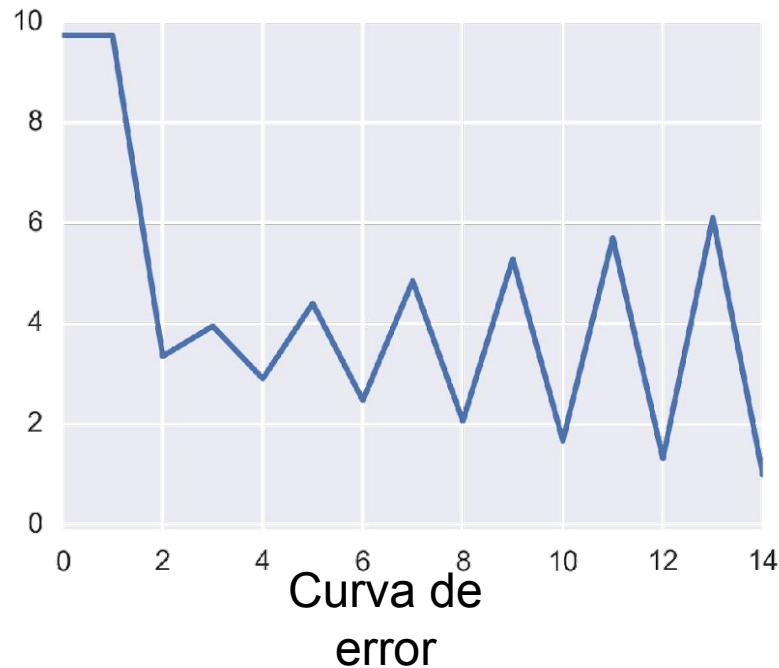
LR $\epsilon=1/8$, 15 pasos



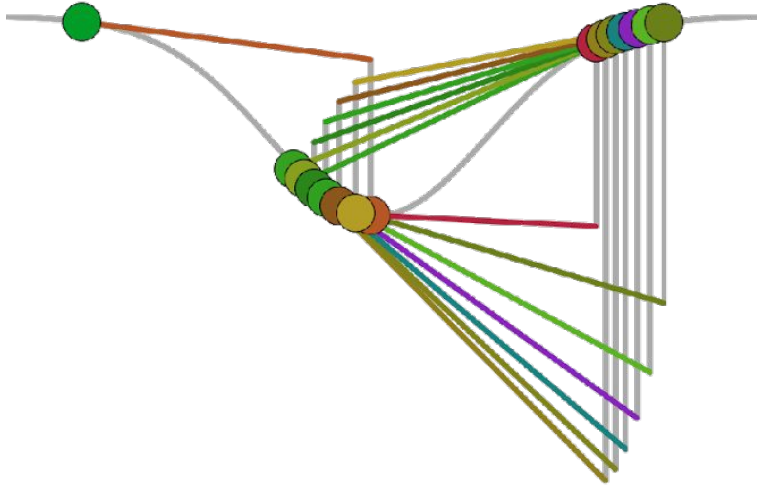
Learning Rate [LR] (tasa de aprendizaje) Constante v



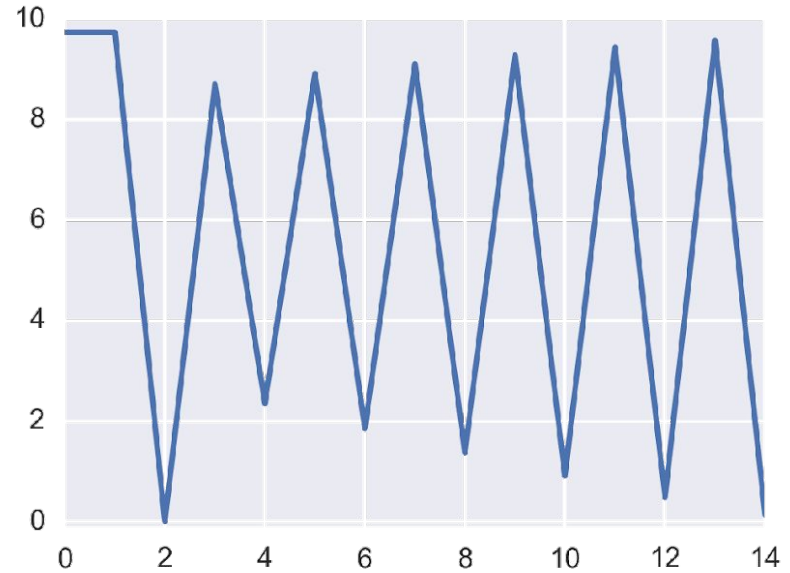
LR $\varepsilon = \frac{1}{2}$, 15 pasos



Learning Rate [LR] (tasa de aprendizaje) Constante

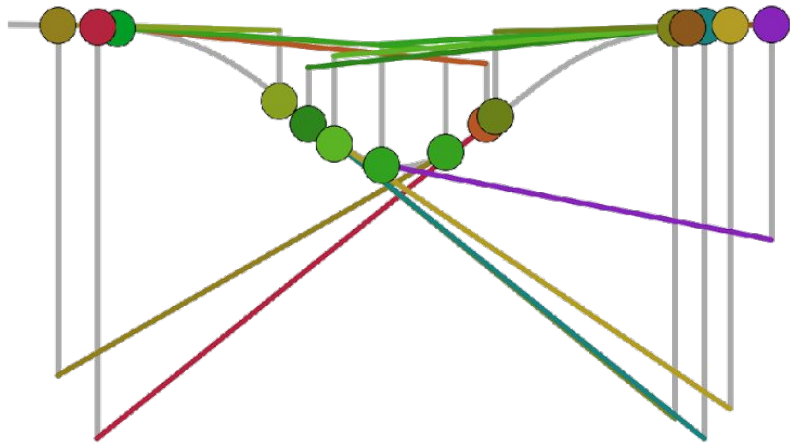


LR $\epsilon = \frac{3}{4}$, 15 pasos

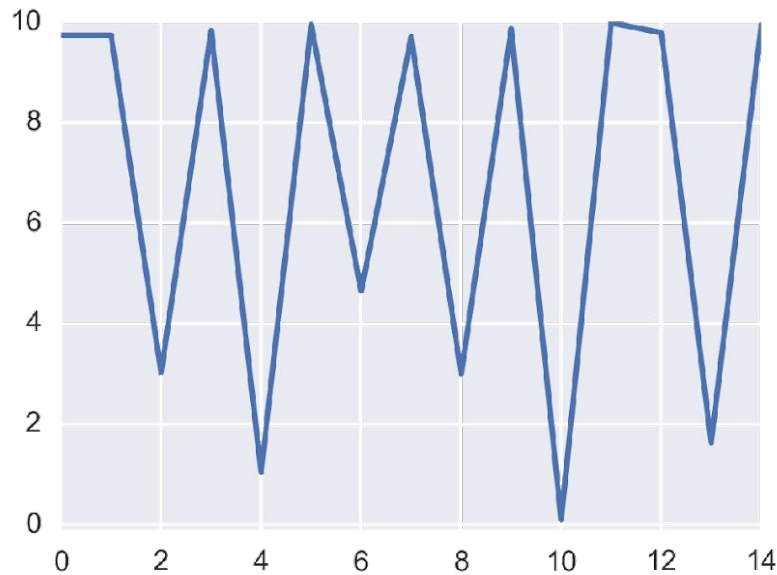


Curva de
error

Learning Rate [LR] (tasa de aprendizaje) Constante



LR $\epsilon=1$, 15 pasos



Curva de
error

Agenda

1. Optimizadores basados en cálculos de gradiente
2. Error Landscape
3. Learning Rate
4. **Schedules**
5. Estrategias de actualización de pesos
6. Acelerando convergencias
7. Learning rate adaptativo
8. ¿Cómo elegir?
9. Bibliografía



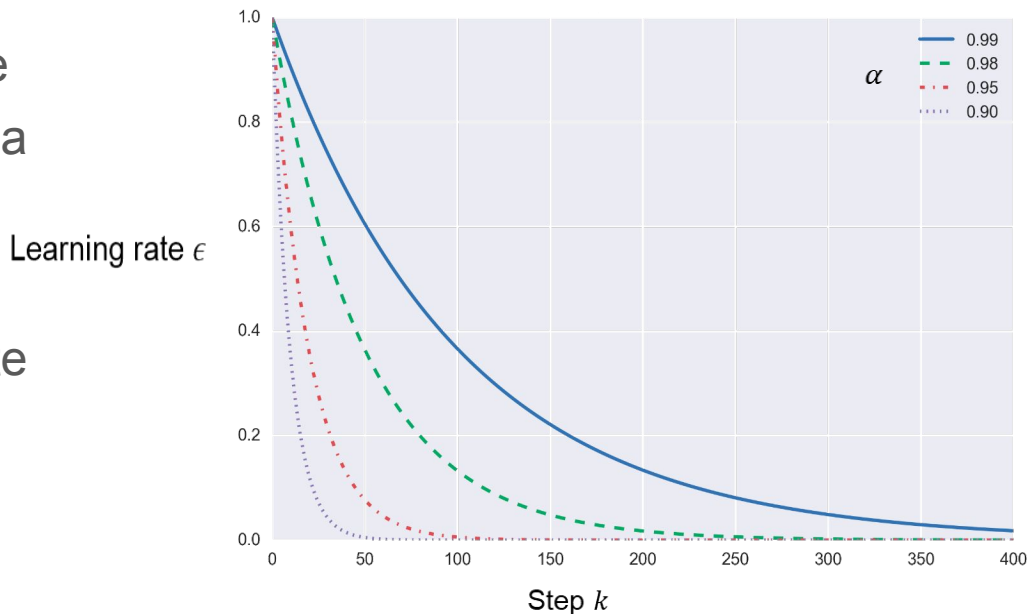
Ajustando el Learning Rate

- Un learning rate muy grande o muy pequeño puede llevar a una mala performance del modelo.
- En general, el learning rate es tuneado utilizando grid search u otras técnicas para optimización de hyper parámetros.
- No siempre es fija:
 - Algunos algoritmos de optimización adaptan el learning rate durante el entrenamiento para mejorar el performance:
 - art vs science

Decaimiento Exponencial (Exponential Decay)

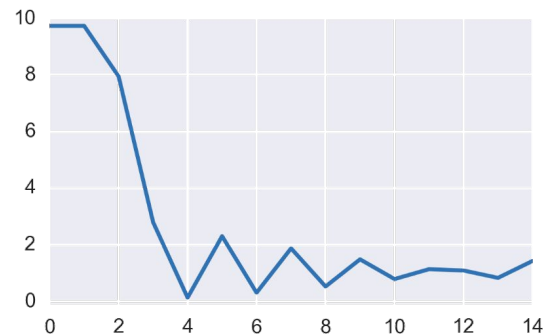
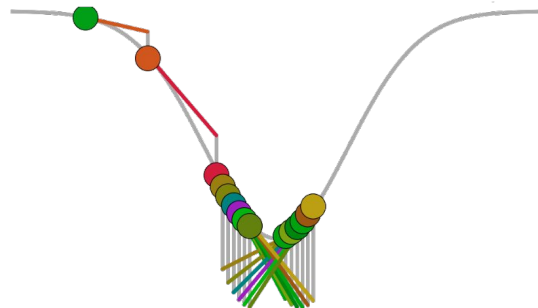
- Empezás con un learning rate **alto** para que el modelo pueda progresar rápidamente.
- **Gradualmente se va disminuyendo** el learning rate (exponencialmente):

○ $\epsilon_k = \epsilon_0 \alpha^k$

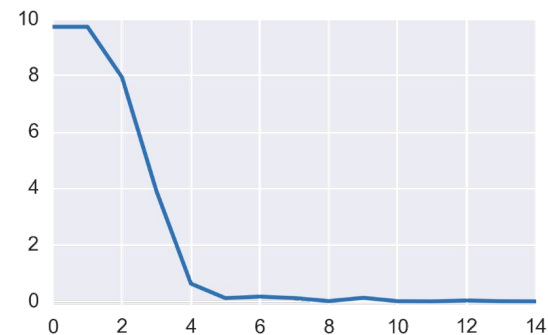
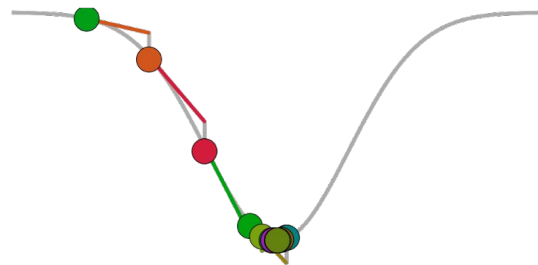


Decaimiento exponencial vs. LR constante

LR constante



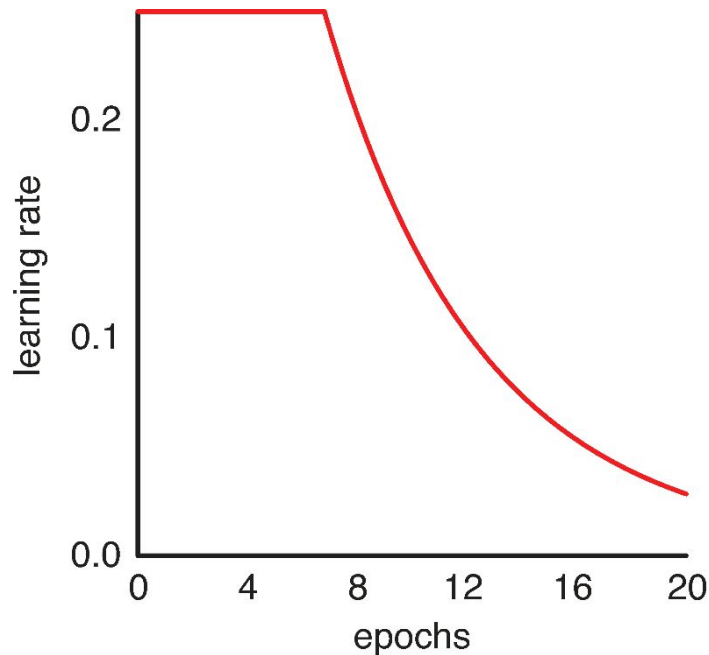
Decaimiento exponencial



Error curves

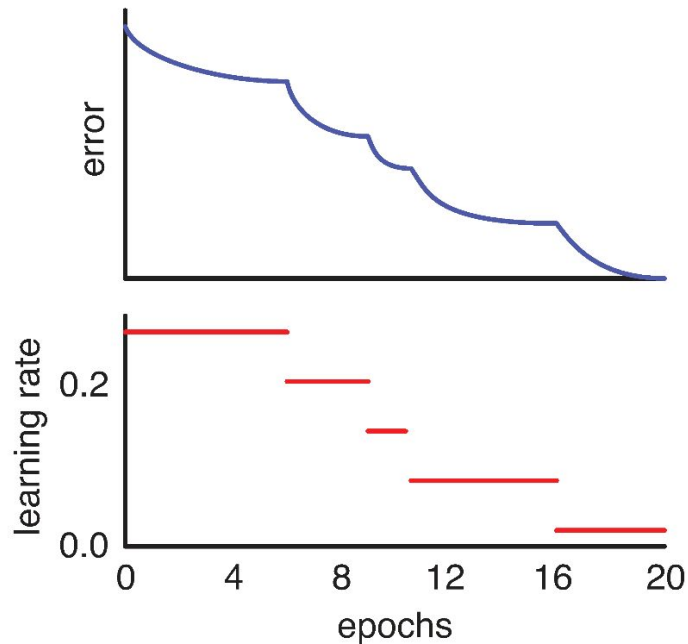
Decaimiento exponencial retrasado

- LR constante durante las primeras iteraciones
 - Mantenemos un LR mas grande para avanzar más rápido en las primeras iteraciones
 - lejos del resultado
- Después de un número fijo de iteraciones, comienza el decaimiento exponencial para ser más precisos.



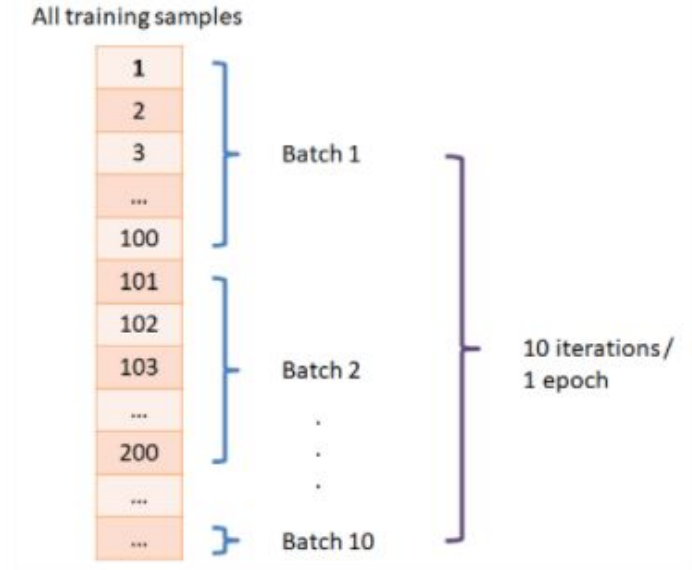
Decaimiento basado en el error

- Ajustamos el LR basándonos en el error
- Disminuimos el LR a medida que el error disminuye
 - Menos error, más precisos
- Útil cuando no hay información que sugiera algún schedule específico.



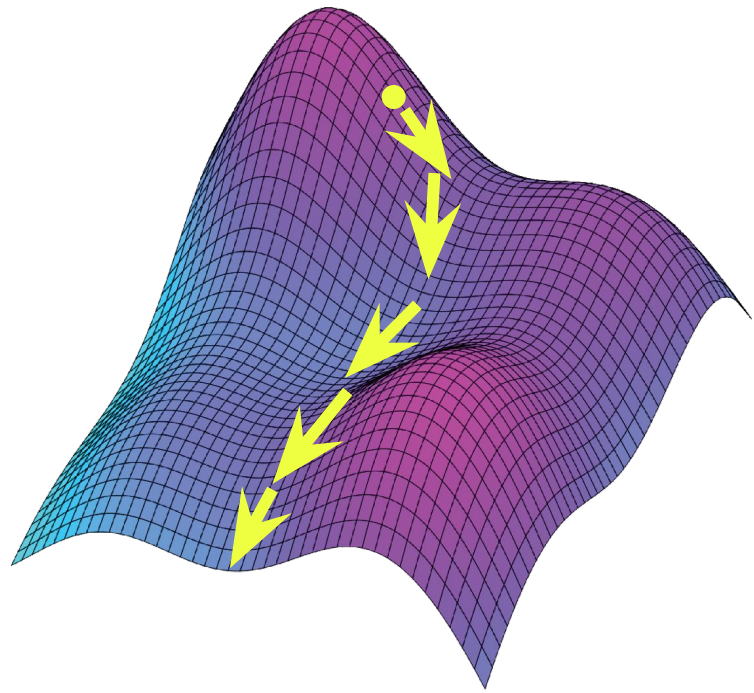
Agenda

1. Optimizadores basados en cálculos de gradiente
2. Error Landscape
3. Learning Rate
4. Schedules
- 5. Estrategias de actualización de pesos**
6. Acelerando convergencias
7. Learning rate adaptativo
8. ¿Cómo elegir?
9. Bibliografía



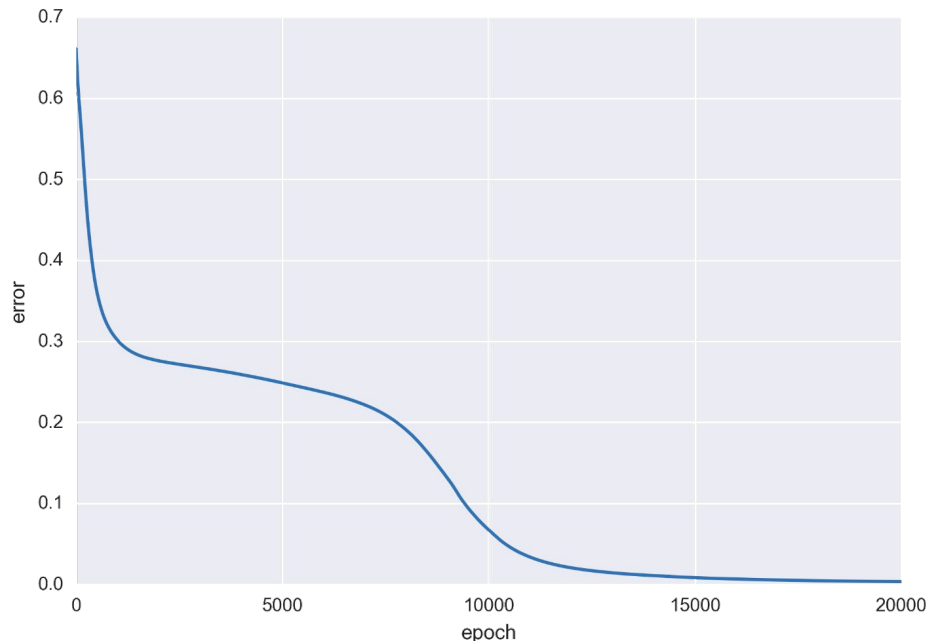
Estrategias para actualizar pesos

- Variaciones del descenso del gradiente:
 - Batch
 - Estocástico
 - Mini-batch
- La diferencia principal es la cantidad de datos utilizados



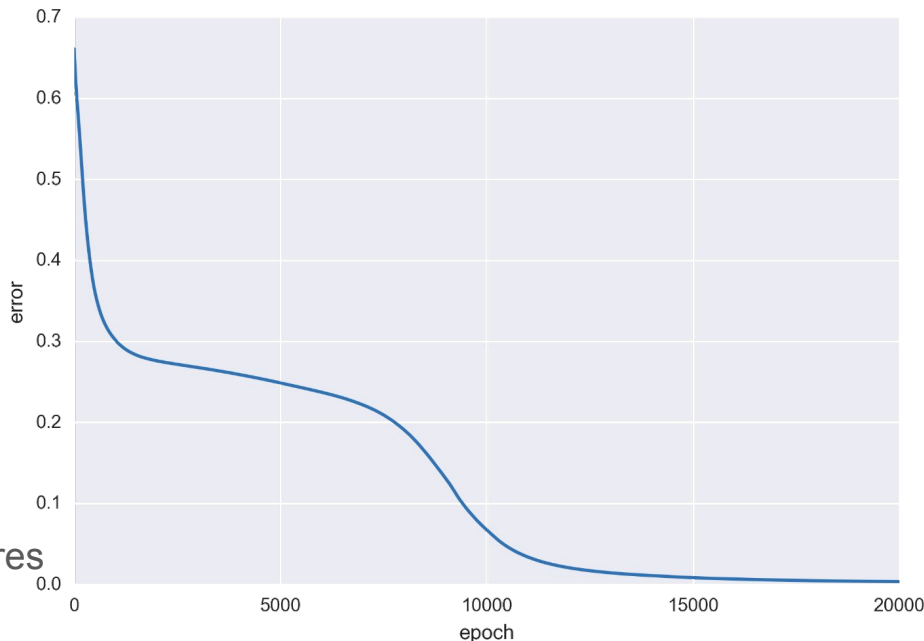
Batch (BGD)

- Se utiliza el dataset entero para calcular el gradiente de la función de costo
- Se actualizan los parámetros del modelo un vez por epoch (Ej. utilizando el promedio)
 - epoch: cada vez que el modelo ve todo el dataset
- Resultados confiables y estables
- Garantizado converger a un mínimo (global si la función es convexa)



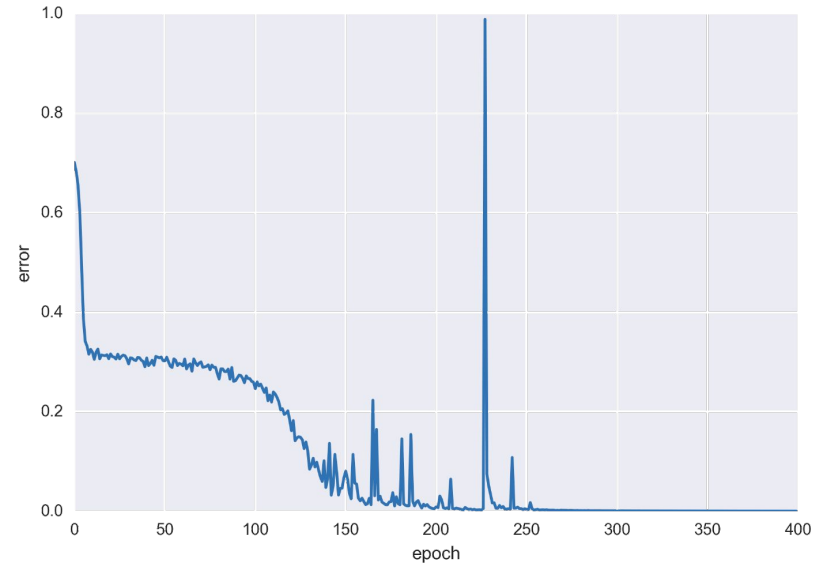
Batch (BGD)

- Curvas de errores suaves
- Poco sensible a outliers
- Costoso:
 - Computacionalmente
 - Memoria
- Puede sufrir de lenta convergencia
 - si el dataset contiene redundancias o features irrelevantes
- No sirve para online learning
 - update data



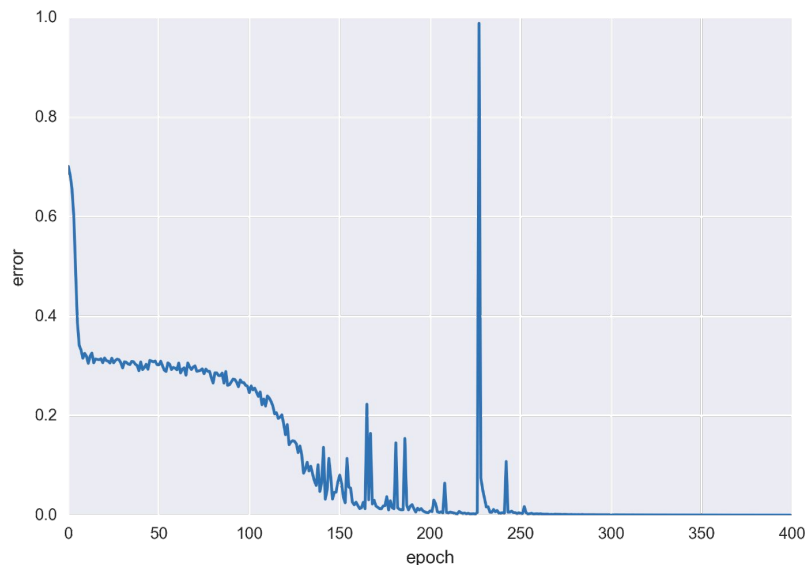
Estocástico (SGD)

- Selecciona el un elemento de entrenamiento de manera **random** para calcular y actualizar los pesos.
- Se repite por cada elemento del dataset hasta converger



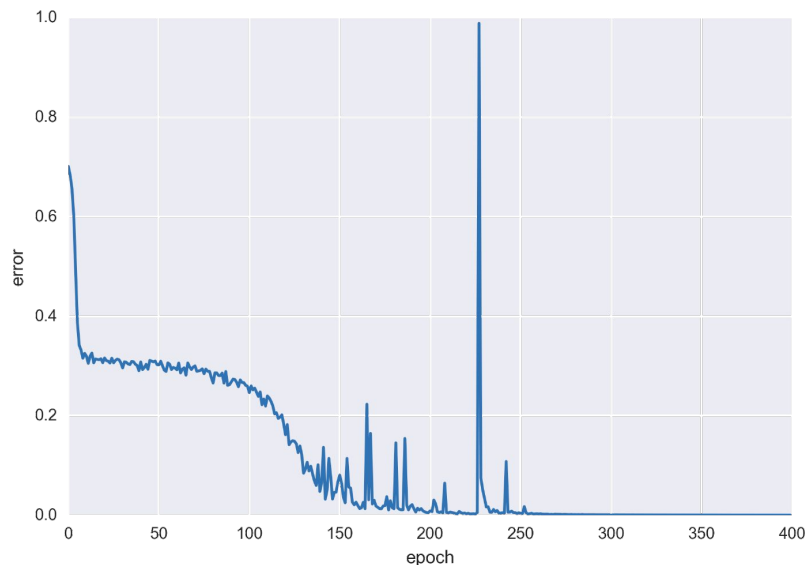
Estocástico (SGD) - Pros

- Menos costoso
 - Computacionalmente
 - Memoria
- Puede converger sin haber visto todos los datos



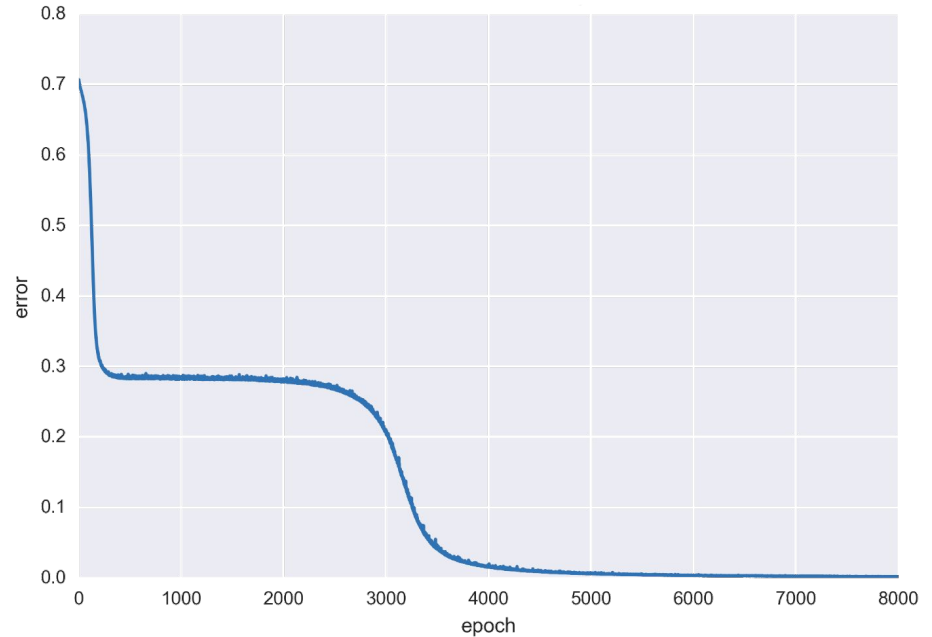
Estocástico (SGD) - Cons

- La selección random del sample hace que la curva sea ruidosa.
- Puede dificultar la selección del criterio de frenada.
- Dificulta la selección del LR



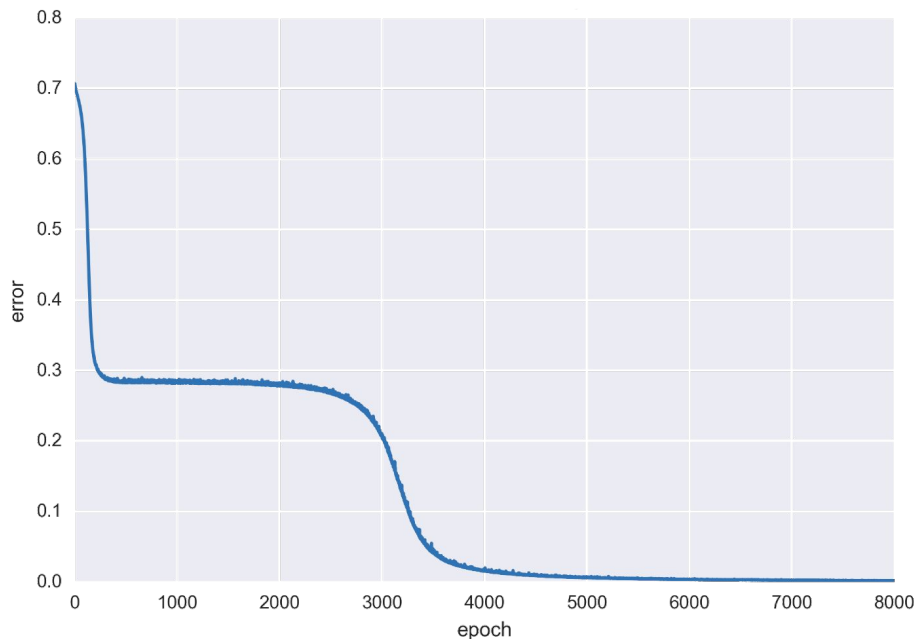
Mini-Batch

- Lo mejor de ambos mundos
- El set de entrenamiento se divide en pequeños batches
 - se calcula el gradiente para cada mini-batch
 - se actualiza el peso



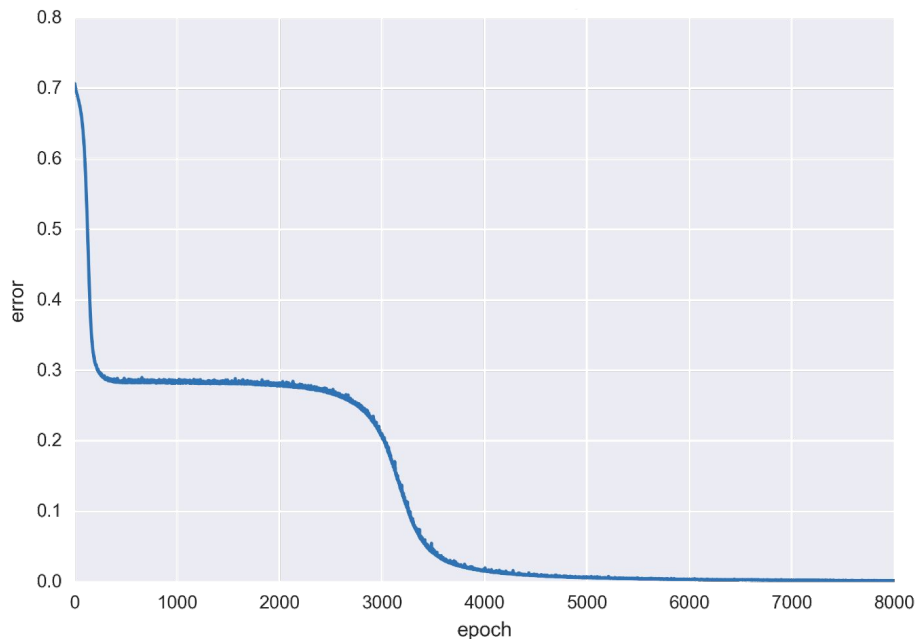
Mini-Batch

- Menos ruidoso que el estocástico
- Graphics processing units (GPUs) puede procesar mini-batch en paralelo
- El más utilizado
- El tamaño del batch impacta la convergencia



Mini-Batch

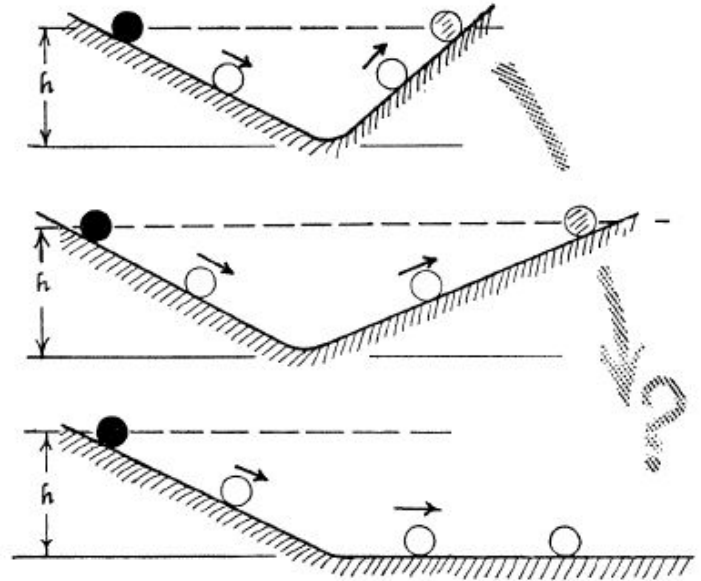
- Un batch **grande**
 - tarda más en calcularse
 - es más preciso.
- Un batch más **pequeño**
 - es más ruidoso
 - actualiza mas seguido
 - puede converger más rápido.



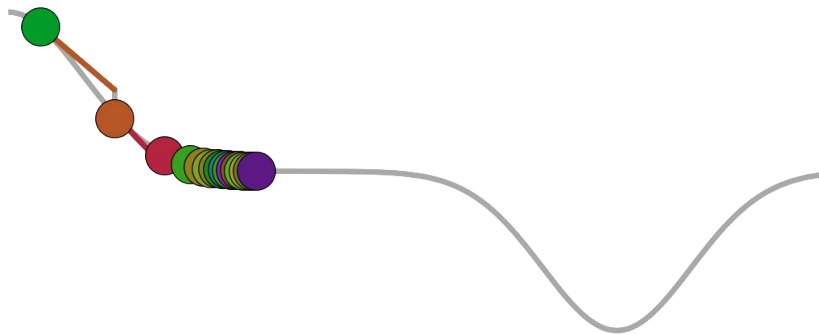
¿Qué pasa cuando en mini-batch, el batch-size es igual a 1?

Agenda

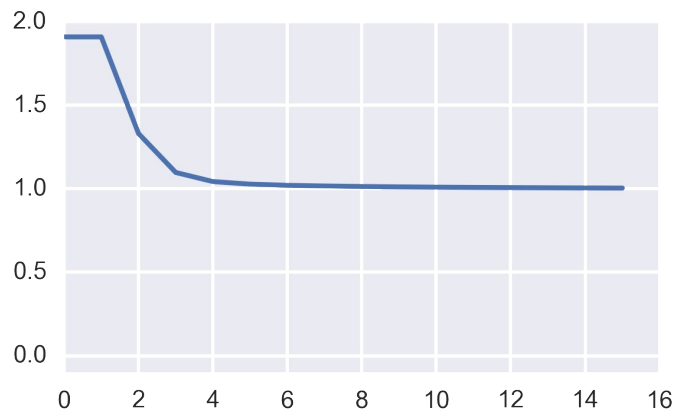
1. Optimizadores basados en cálculos de gradiente
2. Error Landscape
3. Learning Rate
4. Schedules
5. Estrategias de actualización de pesos
6. **Acelerando convergencias**
7. Learning rate adaptativo
8. ¿Cómo elegir?
9. Bibliografía



Convergencia de GD

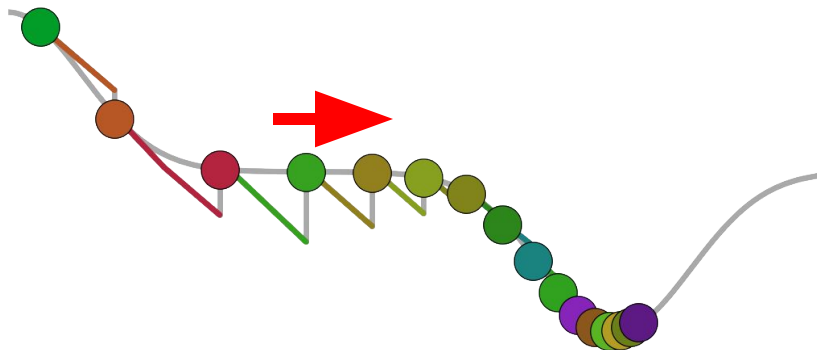


Gradient descent + Exponential decay

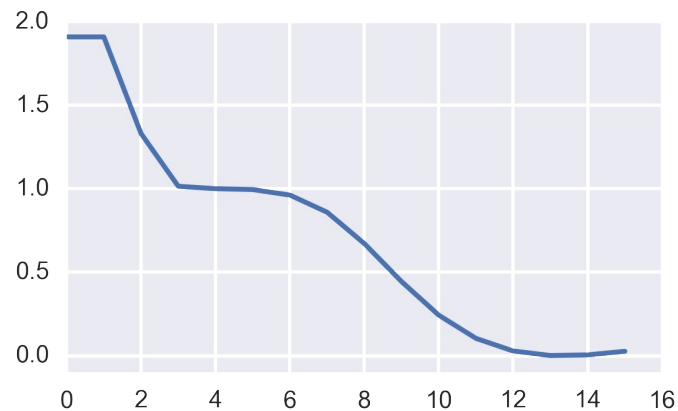


Curva de
error

Convergencia de GD



Momentum



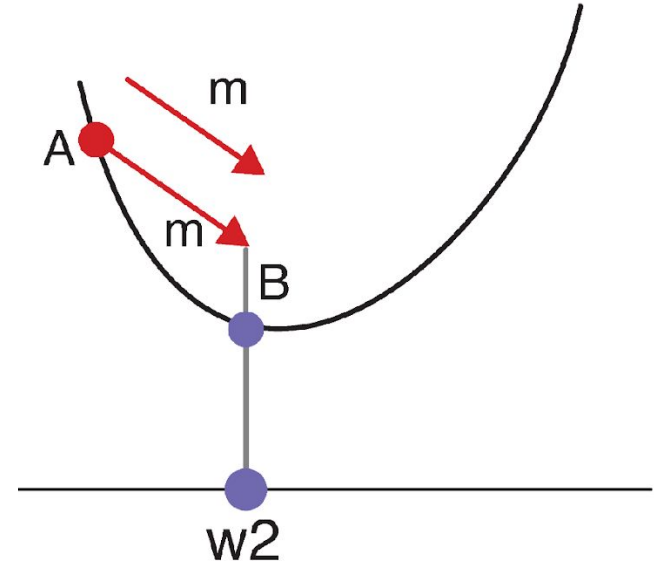
Curva de
error

Acelerando la convergencia

- **Momentum** se utiliza para acelerar la convergencia de GD
- **Idea:** Agregar una fracción del gradiente anterior al actual (**inertia**).
 - Ayuda a suavizar los updates y pasar llanuras y sillas de montar

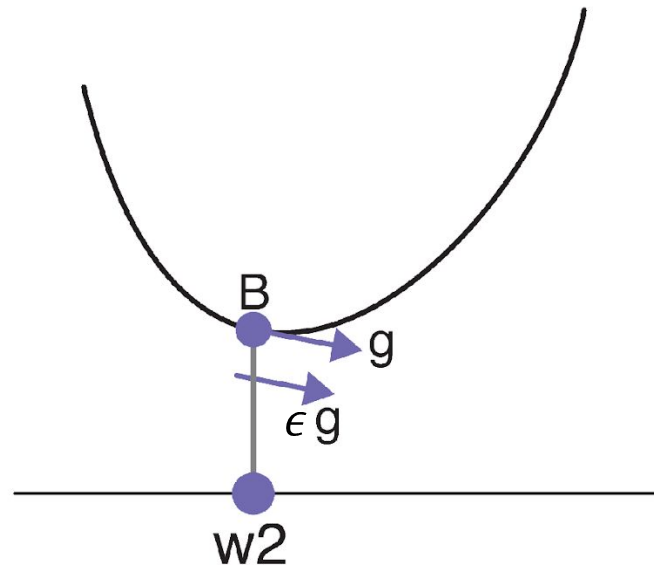
Momentum

- En el punto B, miramos atrás el cambio que aplicamos al punto A (**momentum**).



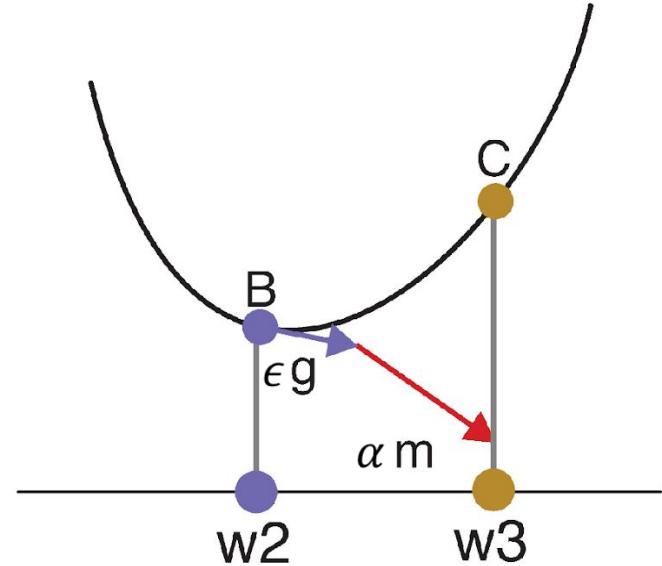
Momentum, Part II

- En el punto B, miramos atrás el cambio que aplicamos al punto A (**momentum**).
- Utilizamos backpropagation para encontrar el gradiente en el punto B (y lo escalamos por ϵ)



Momentum

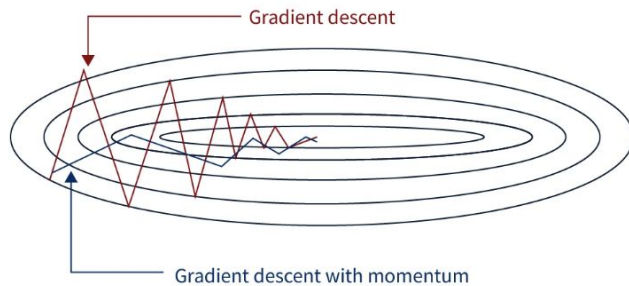
- En el punto B, miramos atrás el cambio que aplicamos al punto A (**momentum**).
- Utilizamos backpropagation para encontrar el gradiente en el punto B (y lo escalamos por ϵ)
- Agregamos el gradiente escalado al momentum escalado (α , entre 0 y 1).



Formal Definition

- El momentum introduce un parámetro de velocidad
- La velocidad acumula el gradiente negativo
 - $\mathbf{v}_t = \alpha \mathbf{v}_{t-1} - \varepsilon \nabla f(\mathbf{x}_t)$
- Descenso de gradiente con updates de momentum

- $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_t$



- α
 - a mas grande α , mas importan los pasos previos en el update
 - suele usarse entre .5 y .99
 - también puede ser adaptado en el tiempo (como ε)
 - aunque es menos importante

Agenda

1. Optimizadores basados en cálculos de gradiente
2. Error Landscape
3. Learning Rate
4. Schedules
5. Estrategias de actualización de pesos
6. Acelerando convergencias
7. **Learning rate adaptativo**
8. ¿Cómo elegir?
9. Bibliografía



Learning Rates Adaptativos

- Learning rate → hyper parámetro difícil de setear
- Impacta significativamente al modelo
- El momentum mitiga el impacto pero introduce otro hyper parámetro
- Idea: **adapting learning** rates para cada peso durante el entrenamiento

AdaGrad

- Adaptive gradient learning algorithm
 - Cambia individualmente el tamaño del gradiente para cada peso
- **Scale factor:**
 - Inversamente proporcional a la raíz cuadrada de la suma de los valores históricos

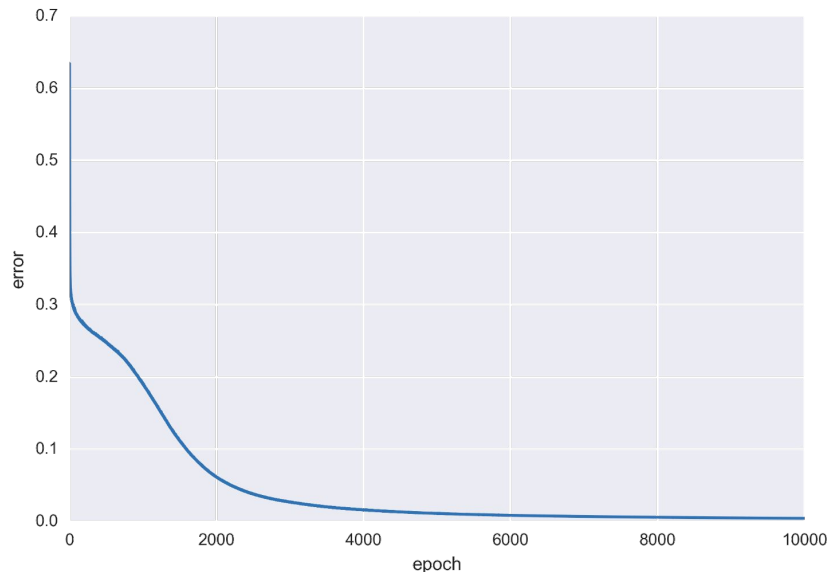
$$\mathbf{r}_t = \mathbf{r}_{t-1} + \nabla f(\mathbf{x}_t) \odot \nabla f(\mathbf{x}_t)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\epsilon}{\delta + \sqrt{\mathbf{r}_t}} \odot \nabla f(\mathbf{x}_t)$$

AdaGrad update rule

AdaGrad

- Mejor progreso en direcciones de poca curvatura
- Permite progreso en regiones de mucha curvatura
- **Problema:** la suma acumulada de $\mathbf{r}$ se agranda y los gradientes se hacen insignificantes (se acercan al cero).



Adadelta and RMSprop

- Idea:
 - usa **decaying sum** de gradientes en vez de los valores acumulados.
 - Valores recientes tienen más influencia que los antiguos
- Suele converger más rápido que AdaGrad
- Contra: Se introduce un nuevo parámetro (γ) que define cuanto achicar el gradiente.

Adaptive Moment Estimation (ADAM)

- Elevar al cuadrado el gradiente

$$\mathbf{s}_t = \beta_1 \mathbf{s}_{t-1} + (1 - \beta_1) \nabla f(\mathbf{x}_t)$$

- es útil

$$\mathbf{r}_t = \beta_2 \mathbf{r}_{t-1} + (1 - \beta_2) \nabla f(\mathbf{x}_t) \odot \nabla f(\mathbf{x}_t)$$

- matemáticamente positivo

(perdemos información del signo)

- ADAM mantiene una segunda suma de gradientes sin elevarlos al cuadrado

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \epsilon \frac{\mathbf{s}_t}{\delta + \sqrt{\mathbf{r}_t}}$$

- El factor de scaling se calcula utilizando ambos

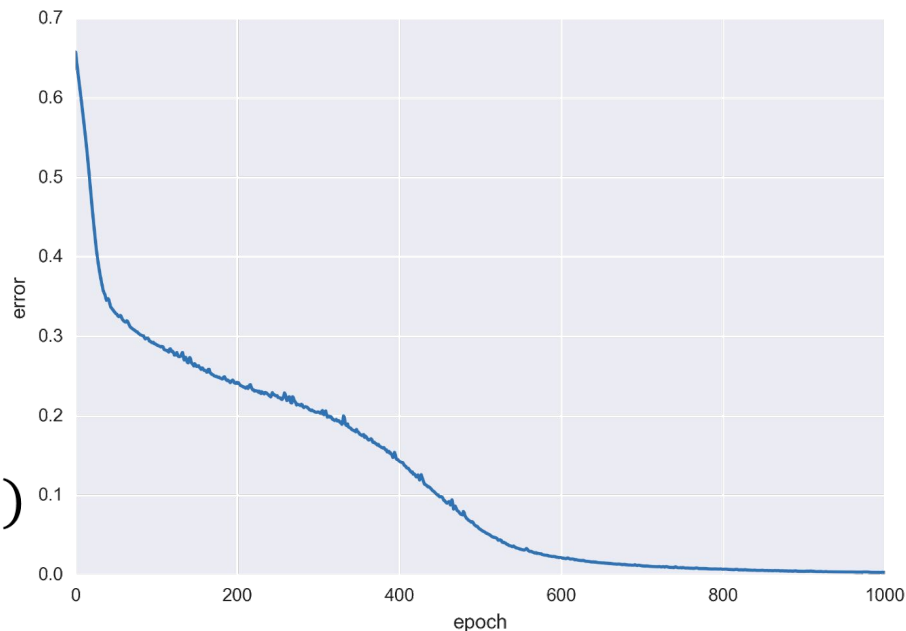
Adaptive Moment Estimation (ADAM), Part III

- Más rápido que AdaGrad o Adadelta
 - muy usado en DL
- Contras:
 - dos hiper-parámetros extra $\beta_1 \beta_2$ (decay para cada suma)

$$\mathbf{s}_t = \beta_1 \mathbf{s}_{t-1} + (1 - \beta_1) \nabla f(\mathbf{x}_t)$$

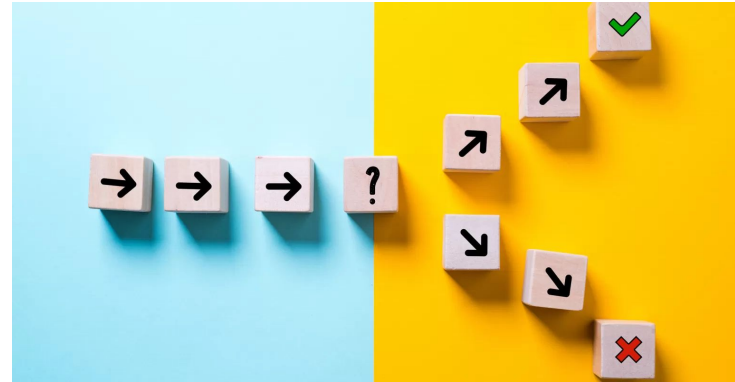
$$\mathbf{r}_t = \beta_2 \mathbf{r}_{t-1} + (1 - \beta_2) \nabla f(\mathbf{x}_t) \odot \nabla f(\mathbf{x}_t)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \epsilon \frac{\mathbf{s}_t}{\delta + \sqrt{\mathbf{r}_t}}$$



Agenda

1. Optimizadores basados en cálculos de gradiente
2. Error Landscape
3. Learning Rate
4. Schedules
5. Estrategias de actualización de pesos
6. Acelerando convergencias
7. Learning rate adaptativo
8. **¿Cómo elegir?**
9. Bibliografía



No Free Lunch

- No existe el “mejor” optimizador.
 - No puede haber un optimizador óptimo para todas las situaciones (**no free lunch theorem, 1997**).
- Podemos encontrar el mejor optimizador para una red específica y una combinación de datos
 - evaluando múltiples algoritmos e hyper parámetros

Bibliografía

- [Deep Learning](#), Ian Goodfellow and Yoshua Bengio and Aaron Courville - Capítulo 4.3.1 y 8.3.1
- Neural Networks and Deep Learning. Charu C. Aggarwal 3.5